CSC-5661
Lab 3
Robert Myers jr.

1. **Describe how you rewarded the agent (4.1) and how you conceptualized the state (4.2).**

   I did a few things to attempt to reward the agent on a few factors. I rewarded based on if it crashed or finished, I also rewarded if it went further on the x axis, so it encouraged the model to make more progress going east.

2. **In your simulations in 3.2. how many turns does it take on average for the agent to cross the finish line with random actions? How often did the agent cross the finish line with the unmodified racetrack? How will the modification we made to the racetrack interact with the reward function to enhance or impede the agent's ability to learn?**

   When doing random actions with the crash resetting to the beginning line. The car would crash around 30,000 times and took around 2000 steps on average to cross the finish line. Without the modified track it would crash all the time and end episodes quickly, so us modifying it would allow us to have the model learn more about the different possible states its in and how to progress. The bad thing is that the model will be learning with the fact that it can always teleport back to the finished line so it might learn to drive into an edge instead of learning to avoid the edges.

3. **Consider the process of training your agent in 4.3. For each of the algorithms and each of the reward functions (mine and yours) how many episodes did you train your agent for? How many turns does each episode take as training progresses? Why do you think there might be a relationship between training time and episode length? Provide visualizations that show the relationship between episode and episode length, average return, and success rate over training. Which algorithm is more sample efficient?**
   So I did only my reward function as for some reason the provided one was taking a really long time to run on Monte Carlo. But I trained an agent on all 3 algorithms for 3000 episodes. Overtime the number of steps decreases as the agents are learning and finishing the racetrack faster. We can see below a strong correlation between average reward and the episode lengths, where the lower the episode length the higher the reward which makes sense as my reward is rewarding based on if a car hits the finish line or crashes.

CSC-5661
Lab 3
Robert Myers jr.

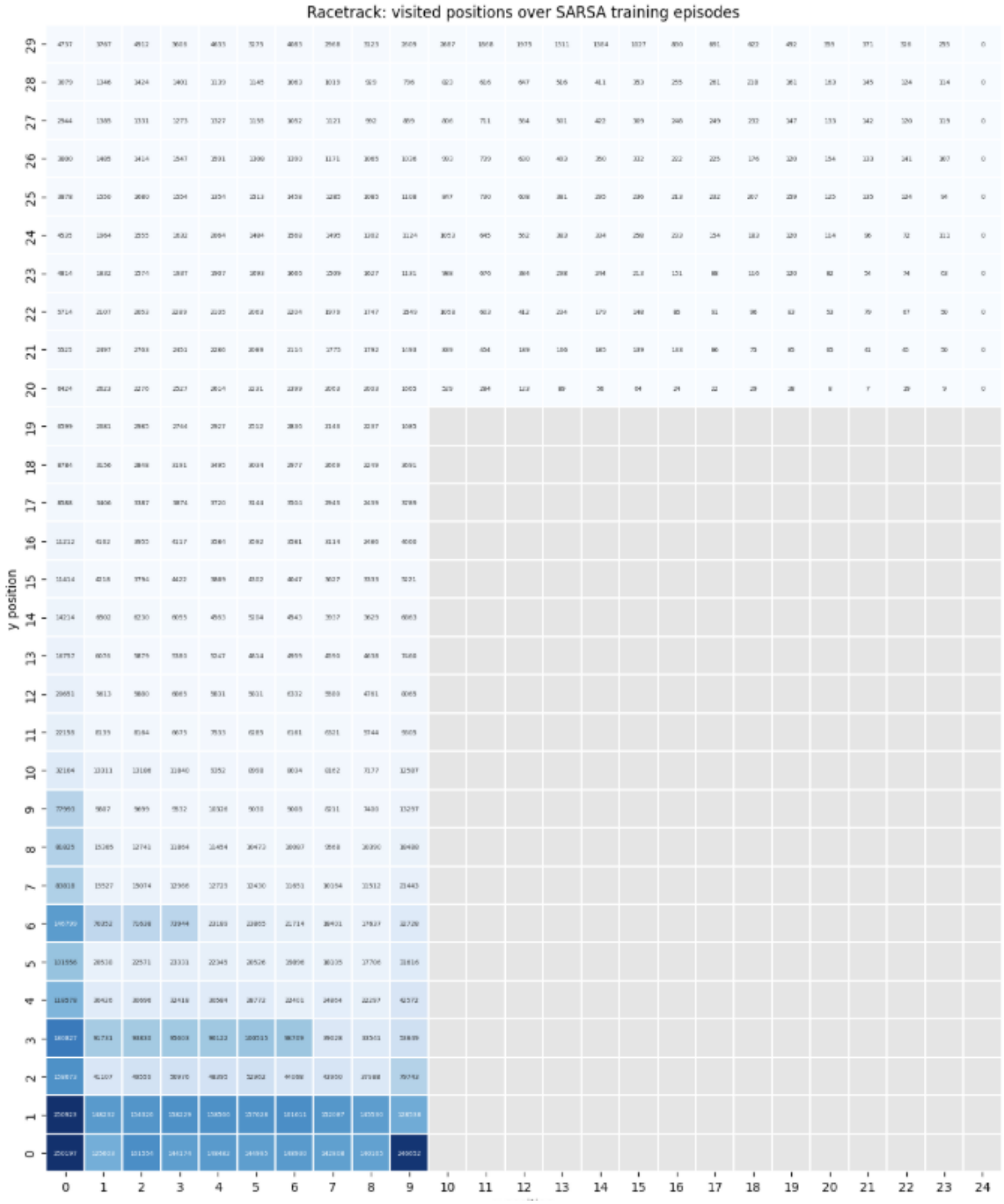
Algorithm	Average Reward	Average Episode Length	Number of Crashes	Total Number of steps
Monte Carlo	-129.1686	310.2694	218034	1551347
Sarsa	-234.4431	486.7798	332394	2433899
Expected Sarsa	-221.6175	463.2812	314838	2316406

It seems like Monte Carlo is the most sample efficient due to the low number of steps.

NOTE: I find these results a little off, the first time running through the lab Monte Carlo was the less efficient which I would imagine is true as it has to do the whole episode twice. Maybe I had stale variable or accidentally changed something during debugging but I think these results are off.

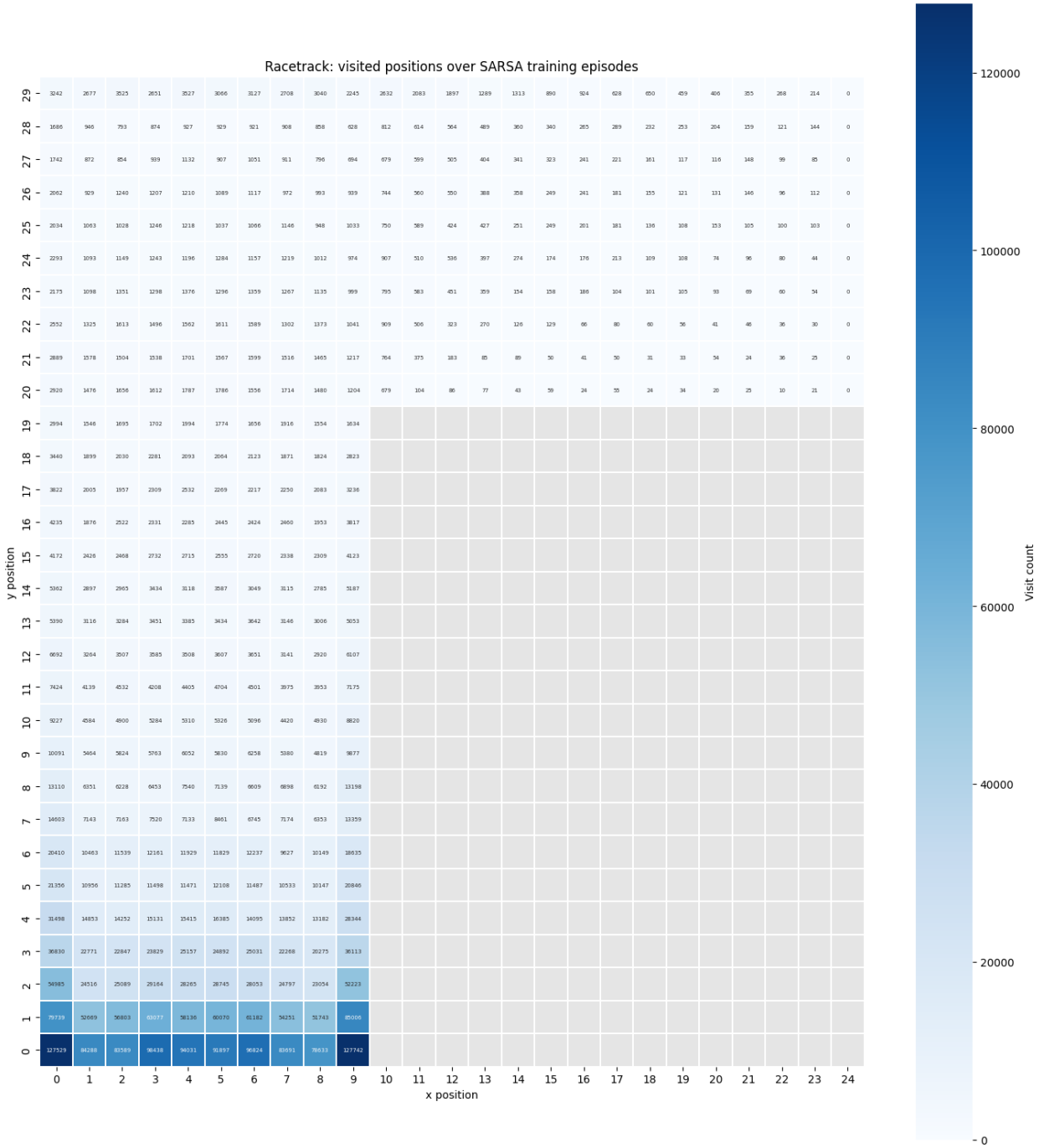
4. **Choose two agents (two reward function/algorithm combinations). Compare the learned policy of these two agents. Specifically, create and provide a visualization of what each agent chooses to do at each location of the track.**
   Expected Sarsa with provided rewards
   crashed: 1516279
   average_episode_length: 3688.2295
   average_reward: 100.22358890708198

CSC-5661
Lab 3
Robert Myers jr.



Expected SARSA with my reward
crashed: 536511
average_episode_length: 1683.926
average_reward: 51.70956195151718

CSC-5661
Lab 3
Robert Myers jr.



CSC-5661
Lab 3
Robert Myers jr.

5. **How do the initial Q-values compare to the learned Q-values for the two algorithms?**

   Initial Q-values are set to zero as place holders, whereas the learned Q-values vary widely based on the reward functions that are defined. Both are structurally similar to the Q map data structures I used, just the magnitude of the reward is different.

6. **What effect (if any) do γ and ε have on the performance of the racetrack agent?**
   Epsilon I found in experimentation was great to have high in the beginning but needed to be lowered overtime so that the agent would try to use the better-known paths, similar to what you do to learning rate with DNN models. Lowering my y from around 0.85 would usually perform worse as it wasn't relying on the older paths and their outcomes.
7. **Describe how you modified the training process to damage the learning of your agent. For each failure case explain what changed, how learning was impacted, and hypothesize how your changes impacted learning. Which algorithm was more robust, and what evidence supports your conclusions?**
   For my first modification I added a chance for wind to blow the car in the track causing the average episode length to be 3463.615. I was able to fix this by adding an incoming wind flag that the agent could use to determine when it needed to drive to counter act the wind bringing the average episode length to be 2277.3555. The second modification I did to make it worse was to make the epsilon a constant 0.9. This made the choices mostly random as it was usually exploring, this resulted in an average episode length of 4892.1305.

CSC-5661
Lab 3
Robert Myers jr.
AI usage.

I used AI in a couple spot areas. Once I created my initial SARSA Algorithm and was happy with it I had claude code create the expected sarsa

Prompt: I want you to create me an expected SARSA class with the given SARSA class that I have. Keep the changes small and keep the environment itself the same.

[Pasted my SARSA cell]

I also used it at the end to convert my Monte Carlo to a function call then I had it make a benchmark cell I used to do all the runs in parallel.

Prompt: Can you convert the montecarlo env to a function and create cells at the bottom to run each environment(Monte Carlo, SARSA, Expected Sarsa) with no wind in parallel, and both my reward and provided. do 3000 episodes and print the heatmap and results of each.